

```

Wed Apr 1 14:36:06 2026
+-----+
| NVIDIA-SMI 580.105.08                  Driver Version: 580.105.08      CUDA Version: 13.0     |
+-----+-----+-----+-----+-----+-----+
| GPU  Name                Persistence-M   Bus-Id        Disp.A         Volatile Uncorr. ECC  |
| Fan  Temp   Perf          Pwr:Usage/Cap     Memory-Usage   GPU-Util  Compute M. |
|                                           MIG M.       |
+-----+-----+-----+-----+-----+-----+
|   0   Tesla T4              Off          00000000:00:04.0 Off           |               0      |
| N/A   41C    P8              9W / 70W           0MiB / 15360MiB      0%          Default  |
|                                           N/A              |
+-----+-----+-----+-----+-----+-----+
|   1   Tesla T4              Off          00000000:00:05.0 Off           |               0      |
| N/A   40C    P8              9W / 70W           0MiB / 15360MiB      0%          Default  |
|                                           N/A              |
+-----+-----+-----+-----+-----+-----+
+-----+
| Processes:                                |
| GPU   GI    CI          PID    Type    Process name                        GPU Memory |
| ID     ID     ID                     |           |                      Usage      |
+-----+-----+-----+-----+-----+-----+
| No running processes found              |           |                      |
+-----+

```

Figura 1: Diagnóstico de hardware mediante la utilidad NVIDIA System Management Interface (nvidia-smi).

2. Análisis de Coalescencia y Transacciones de Memoria

Contexto:

En el Experimento 2, observamos una degradación del Ancho de Banda Efectivo de 2.37 GB/s (Stride 1) a 1.90 GB/s (Stride 32).

Análisis del Desperdicio (96.8%): Cuando la GPU ejecuta un acceso con Stride 32, cada hilo de un Warp (32 hilos) solicita un dato que está separado por 32 posiciones de memoria. Debido a que el hardware de la Tesla T4 siempre recupera líneas de 128 bytes (una transacción de memoria completa), y cada hilo solo necesita 4 bytes (un float), la GPU se ve obligada a realizar 32 transacciones distintas para satisfacer a un solo Warp. En cada transacción de 128 bytes, el hilo toma sus 4 bytes y el hardware descarta los 124 bytes restantes. Esto supone que el 96.8% del tráfico movido por el bus de datos es "basura" informativa.

Pregunta (Tiempo vs. Ancho de Banda Útil): El tiempo de ejecución apenas varía significativamente (de 14.1 ms a 17.7 ms) porque el volumen total de instrucciones no cambia, pero el Ancho de Banda Útil cae en picado porque el bus está saturado con transacciones desperdiciadas. La GPU está trabajando al máximo de su capacidad física de transferencia, pero la mayoría de esos bytes no se usan para el cálculo, lo que resulta en una eficiencia de bus bajísima.

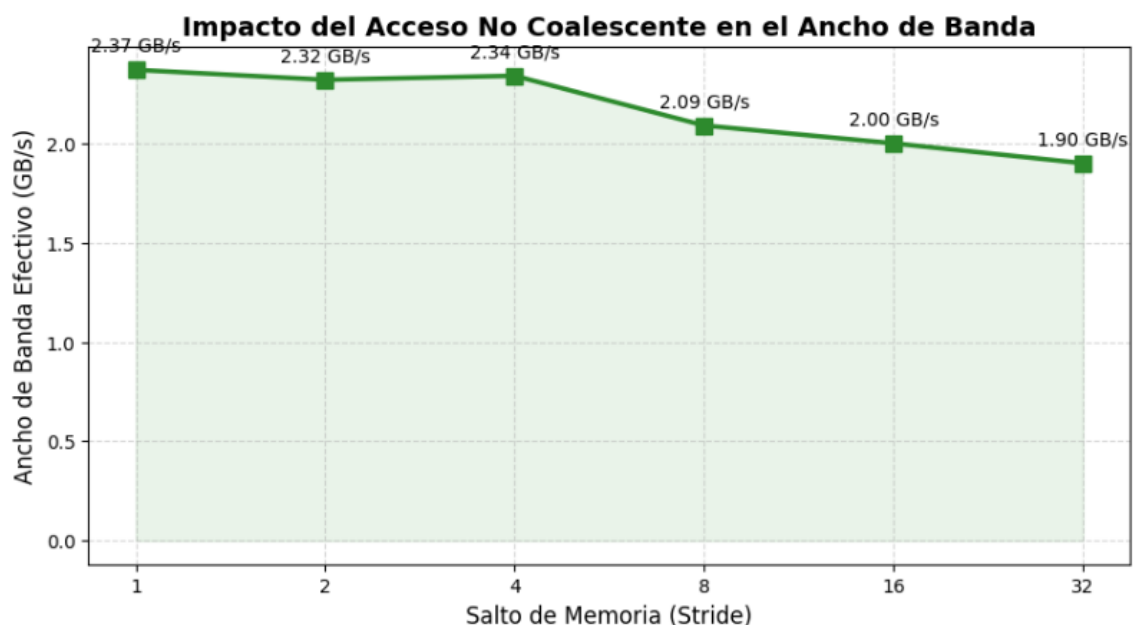


Figura 2: Evolución del Ancho de Banda Útil en función del patrón de acceso a memoria (Stride).

3. Diagnóstico mediante el Modelo Roofline

Contexto: La gráfica muestra una zona de latencia oculta hasta las 10-25 operaciones por byte, seguida de un incremento lineal del tiempo.

Análisis del "Codo": En mi gráfica, el "codo" o punto de inflexión se sitúa cerca de las 50 OPS/Byte.

Memory-Bound: El algoritmo está limitado por la velocidad de la memoria. Añadir más cálculos no aumenta el tiempo porque las ALUs están inactivas esperando datos.

Compute-Bound: El algoritmo está limitado por la potencia de cálculo. Los datos llegan más rápido de lo que las ALUs pueden procesarlos.

Pregunta (Evolución del Codo): Si tuviéramos núcleos el doble de rápidos pero la misma memoria, el codo se movería hacia la DERECHA.

Justificación: Al ser los núcleos más rápidos, el algoritmo procesará las operaciones en la mitad de tiempo, por lo que necesitará todavía más intensidad aritmética (más operaciones por cada dato leído) para llegar a saturar los nuevos núcleos. El algoritmo seguiría siendo Memory-Bound durante un rango más amplio de la gráfica.

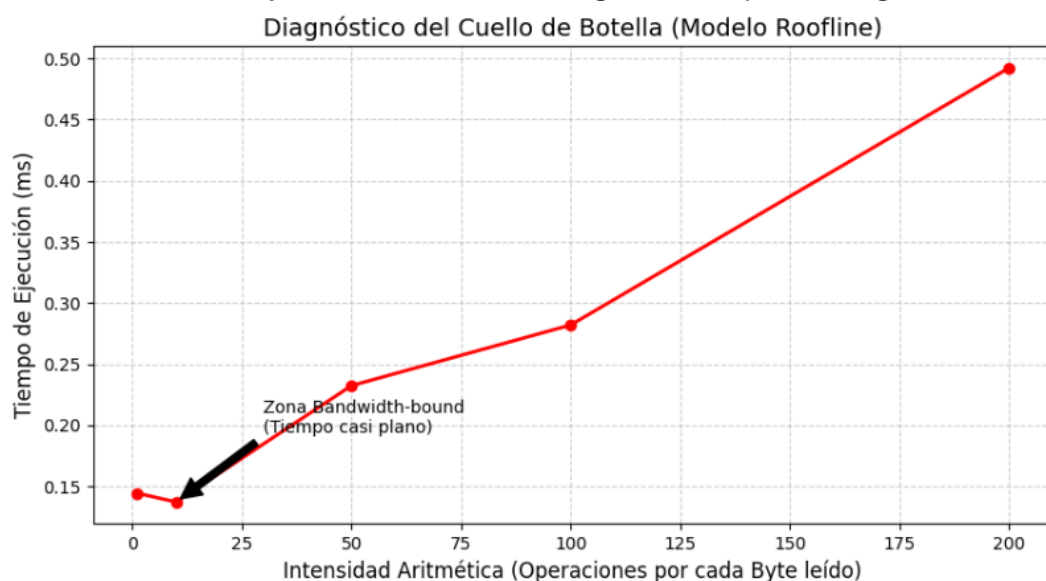


Figura 3: Modelo Roofline empírico detallando la transición entre los estados Bandwidth-bound y Compute-bound.

4. Optimización mediante Shared Memory y Tiling

Contexto: Se obtuvo un Speedup masivo en la reducción cooperativa (de 37.6 ms a 1.7 ms).

Pregunta 1:

(Eficiencia y atomicAdd): El uso de `__shared__` reduce la congestión porque implementa una estrategia de reducción local. En el método de VRAM directa, miles de hilos compiten por la misma dirección de memoria global, provocando una serialización forzada en el bus. Al usar memoria compartida, los hilos colaboran dentro del SM para sumar sus valores en una memoria "on-chip" de bajísima latencia. Al final, solo un hilo por bloque (en lugar de 1024) realiza el `atomicAdd` a la VRAM, reduciendo la presión sobre el bus global en un factor de 1024x.

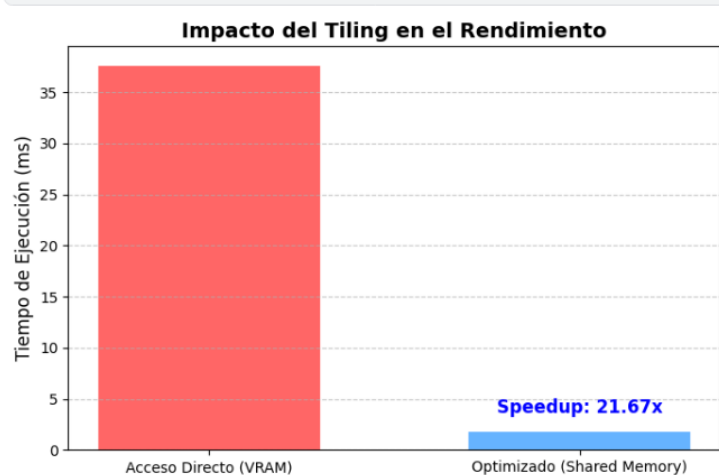


Figura 4: Análisis comparativo de latencia entre el acceso directo a VRAM y la optimización mediante Tiling con Shared Memory.

Pregunta 2:

(Asfixia del SM y Sincronización): Al observar la gráfica de Speedup vs. Ocupación (Imagen 4), el rendimiento cae al usar 1024 hilos debido a la asfixia de recursos y el coste de sincronización.

A 1024 hilos por bloque, el SM agota su Archivo de Registros y la Shared Memory disponible por bloque, lo que impide que otros bloques se planifiquen en el mismo SM (baja ocupación efectiva).

Además, el uso de `__syncthreads()` se vuelve extremadamente costoso: un bloque de 1024 hilos debe esperar a que todos sus hilos lleguen a la barrera. Cuanto más grande es el bloque, más probable es que un solo hilo lento (por divergencia o latencia) detenga a los otros 1023, destruyendo la eficiencia del paralelismo.

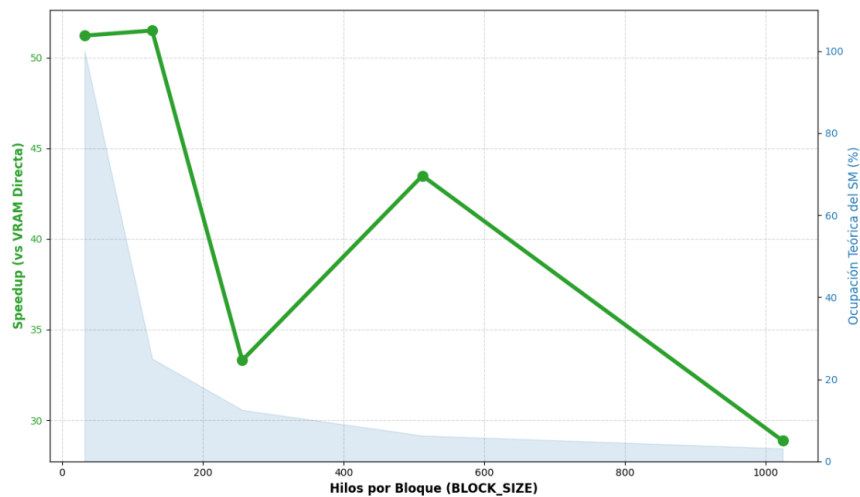


Figura 5: Correlación entre el factor de aceleración (Speedup) y la ocupación teórica del SM según el tamaño de bloque.

5. Conclusión Final: Las 3 Reglas de Oro del Ingeniero HPC

Regla de la Coalescencia (Orden de Acceso):

"Asegura siempre un Stride de 1". Los hilos de un mismo Warp deben acceder a direcciones de memoria contiguas para maximizar el aprovechamiento de cada transacción de 128 bytes y evitar el desperdicio del ancho de banda del bus.

Regla de la Jerarquía (Shared Memory):

"Carga una vez, utiliza muchas". Si un dato va a ser accedido por varios hilos o reutilizado, debe moverse de la VRAM a la `__shared__` memory. Es preferible pagar el coste de una carga inicial coordinada que saturar la VRAM con accesos redundantes y atómicos.

Regla de Configuración (Lanzamiento Óptimo):

"Busca el equilibrio, no el máximo". No siempre 1024 hilos es la mejor opción. Se debe configurar un BLOCK_SIZE (típicamente 128 o 256) que permita una alta ocupación sin asfixiar al SM por falta de registros y sin penalizar el rendimiento por sincronizaciones de barrera excesivamente grandes.

